Lec 5: Knapsack + MDP

Tuesday, September 8, 2026 1:41 PM

CS 6515: Graduate Algorithms

Lecture 5: Knapsack FPTAS and Finite-Horizon MDPs

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW2 is out; due Sep 17. Although slightly long (6 pbs), you don't have to attempt them all.
2. Exam 1 is Sep 22.

2 Where we left off: value-indexed knapsack

There are n items, with positive integer sizes s_i and values v_i, and the knapsack has capacity B. Let

$$V_{\text{sum}} = \sum_{i=1}^{n} v_i.$$

HW-2: Alg: poly(n) · B

Recall the value-indexed subproblem

$$D[i, v] = \text{minimum total size of a subset of the first } i \text{ items having value exactly } v,$$

where an impossible state has value $+\infty$. The recurrence is

$$D[i, v] = \begin{cases} \min\{D[i-1, v],\ s_i + D[i-1, v - v_i]\}, & v \geq v_i, \\ D[i-1, v], & v < v_i. \end{cases}$$

The optimum is the largest v for which $D[n, v] \leq B$. The resulting exact algorithm runs in

$$\mathcal{O}(nV_{\text{sum}}) = \mathcal{O}(n^2 \max_i v_i)$$

$\leq n/\epsilon \Rightarrow n^3/\epsilon$

time. This is *pseudo-polynomial*: it is polynomial in the numerical values v_i, but not necessarily in their binary encoding lengths.

The obstacle. The dynamic program is slow only because its value axis can be enormous. Can we compress that axis while losing only a small amount of objective value?

3 An FPTAS for knapsack

For a maximization problem, a $(1-\varepsilon)$-approximation returns a feasible solution of value at least $(1-\varepsilon)\,\text{OPT}$, where $0 < \varepsilon < 1$. A polynomial-time approximation scheme (PTAS) has polynomial running time for every fixed ε. A fully polynomial-time approximation scheme (FPTAS) has running time polynomial in both the input length and $1/\varepsilon$.

optional

Thm: We output a soln of val $\geq (1-\epsilon)\,\text{opt}_{\text{val}}$ in time $O(n^3/\epsilon)$

First discard items with $s_i > B$, since they can never be selected. If no item remains, return the empty set. Otherwise let

$$V_{\max} = \max_i v_i.$$

An item attaining $V_{\max}$ is feasible by itself, so $V_{\max} \le \mathrm{OPT}$.

Choose the scaling unit

$$K = \frac{\varepsilon V_{\max}}{n}$$

and replace each value by the nonnegative integer

$$\widehat{v}_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

Run the value-indexed knapsack DP using the scaled values $\widehat{v}_i$, and let $\widehat{S}$ be the feasible subset with maximum total scaled value.

Theorem (knapsack FPTAS). For every $0 < \varepsilon < 1$, the scaling algorithm returns a feasible set $\widehat{S}$ such that

$$v(\widehat{S}) \ge (1 - \varepsilon)\,\mathrm{OPT}.$$

Its running time is $\mathcal{O}(n^3/\varepsilon)$ under the unit-cost arithmetic convention used for the pseudo-polynomial DP.

Running time. For every remaining item,

$$\widehat{v}_i \le \frac{v_i}{K} \le \frac{V_{\max}}{K} = \frac{n}{\varepsilon}.$$

Consequently,

$$\sum_i \widehat{v}_i \le \frac{n^2}{\varepsilon}, \qquad \mathcal{O}\left(n \sum_i \widehat{v}_i\right) = \mathcal{O}(n^3/\varepsilon).$$

Thus the scaled table has polynomial size in both the input length and $1/\varepsilon$.

3.1 Approximation guarantee

Rounding down loses less than K on each item:

$$v_i - K < K\widehat{v}_i \le v_i.$$

Let S^* be an optimal solution. Since $\widehat{S}$ maximizes scaled value among all feasible sets,

$$\begin{aligned} v(\widehat{S}) &\ge K \sum_{i \in \widehat{S}} \widehat{v}_i \\ &\ge K \sum_{i \in S^*} \widehat{v}_i \\ &> \sum_{i \in S^*} (v_i - K) \\ &\ge \mathrm{OPT} - nK \\ &= \mathrm{OPT} - \varepsilon V_{\max} \\ &\ge (1 - \varepsilon)\,\mathrm{OPT}. \end{aligned}$$

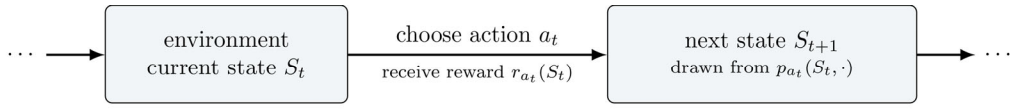
The first inequality translates scaled value back to true value; the second uses optimality for the scaled instance; and the remaining inequalities bound the accumulated rounding loss.

What to notice. The approximation algorithm uses the same dynamic program. We changed only the numbers indexing its states, trading an additive loss of at most nK for a polynomial number of states.

Exercise. For sizes $(6, 3, 4, 2)$, values $(30, 14, 16, 9)$, capacity $B = 10$, and $\varepsilon = 1/2$, compute K and the scaled values. Compare the returned solution with $(1 - \varepsilon)\,\mathrm{OPT}$. Is the guarantee tight on this instance?

4 Finite-horizon Markov decision processes

From deterministic DP to decisions under uncertainty. In knapsack, choosing whether to include an item determined the next subproblem. We now consider decisions whose outcomes are uncertain: an action leads to a *distribution* over possible next states. The dynamic-programming principle remains the same—summarize the future by the value of a smaller state.



Markov decision process. A finite-horizon Markov decision process (MDP) consists of:

- a finite state space $\mathcal{S}$;
- a finite action space $\mathcal{A}$;
- a starting state $s_1 \in \mathcal{S}$;
- transition probabilities $p_a(s, s')$ for every a, s, s', where

$$p_a(s, s') \geq 0, \qquad \sum_{s' \in \mathcal{S}} p_a(s, s') = 1;$$

- an immediate reward $r_a(s) \in \mathbb{R}$ for taking action a in state s;
- a time horizon T.

When action a_t is taken in state S_t, reward $r_{a_t}(S_t)$ is received, and the next state S_{t+1} is drawn according to $p_{a_t}(S_t, \cdot)$.

Definition (policy). A policy specifies which action to take at each step. In full generality, it could depend on the entire history of visited states and chosen actions.

The objective is

$$\max_{\pi} \; \mathbb{E}\left[\sum_{t=1}^{T} r_{a_t}(S_t)\right].$$

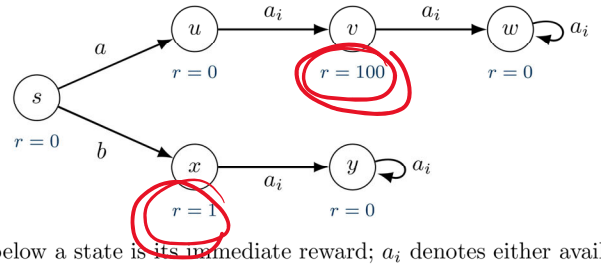
The transition table has $\Theta(|\mathcal{A}||\mathcal{S}|^2)$ entries when it is stored explicitly.

5 Two small examples

5.1 Changing the horizon can change the first action

Imagine a start state with two actions. Action a enters a two-step route that eventually pays reward 100, while action b enters a one-step route that pays reward 1. All transitions are deterministic.

- With one step remaining, neither route may have time to pay.
- With two steps remaining, the short route can be better.
- With three steps remaining, the long route can be much better.

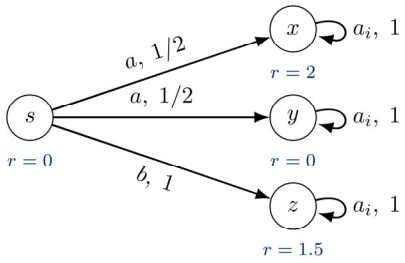


The label below a state is its immediate reward; a_i denotes either available action.

The optimal action can therefore depend on both the current state and the number of steps remaining.

5.2 Expected reward, not best possible reward

From a state s, suppose action a moves with probability $1/2$ to a state paying 2 next step and with probability $1/2$ to a state paying 0. Action b moves deterministically to a state paying 1.5.



With two steps remaining, the continuation values are

$$\frac{1}{2}(2) + \frac{1}{2}(0) = 1 \qquad \text{and} \qquad 1.5,$$

so b is better even though a has the better possible outcome.

Exercise. For the first example, determine an optimal first action and the resulting total reward for horizons $T = 1, 2, 3$. For the second example, do the same for horizons $T = 1, 2$. What distinct lessons do the two examples illustrate?

6 The Bellman recurrence

Definition (optimal value). Let $V^*(s, t)$ be the maximum expected total reward obtainable when the current state is s and exactly t actions remain.

The base case is

$$V^*(s, 0) = 0 \qquad \text{for every } s \in \mathcal{S}.$$

For $t \geq 1$, conditioning on the first action and next state gives

$$V^*(s, t) = \max_{a \in \mathcal{A}} \left\{ r_a(s) + \sum_{s' \in \mathcal{S}} p_a(s, s') V^*(s', t-1) \right\}.$$

This is the *Bellman optimality equation.*

Observation. There is an optimal policy whose action depends only on the current state and the number of steps remaining. The earlier history contains no additional information needed by the Bellman recurrence.

Observation. There is an optimal deterministic policy. At each state and time, choose an action attaining the maximum in the Bellman equation. Randomizing among actions cannot improve on the best action's expected value.

6.1 Bottom-up algorithm

1. Set $V[s, 0] = 0$ for every state s.
2. For $t = 1, 2, \ldots, T$ and every s, evaluate the Bellman expression for each action a.
3. Store both $V[s, t]$ and an action $\text{act}[s, t]$ attaining the maximum.
4. Starting in s_1 with T steps remaining, use $\text{act}[s, t]$ and decrement t after every transition.

Theorem (finite-horizon MDP). An optimal finite-horizon policy can be computed in

$$\mathcal{O}(T|\mathcal{A}||\mathcal{S}|^2)$$

time and $\mathcal{O}(T|\mathcal{S}|)$ space when transitions are represented explicitly.

Running time. There are $T|\mathcal{S}|$ DP states. For each, we consider $|\mathcal{A}|$ actions and sum over $|\mathcal{S}|$ possible next states.

Correctness. Induct on t. The base case is immediate. For the inductive step, every policy chooses some first action a. Conditional on the next state s', the inductive hypothesis says that no continuation beats $V^*(s', t-1)$. Choosing the maximizing first action and then an optimal continuation attains the upper bound.

Exercise. If only the optimal expected value is required, reduce the space to $\mathcal{O}(|\mathcal{S}|)$. Why does recovering the entire policy require storing more information or recomputing values?